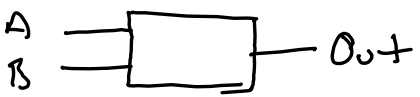
DIGITAL ICs

Logic Gates: AND, OR, NAND, NOR, NOT, XOR



A	B	Out
0	0	0
0	1	1
1	0	1
1	1	1

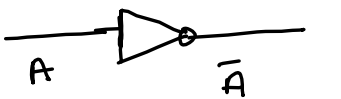
2^4 16 possible output combinations

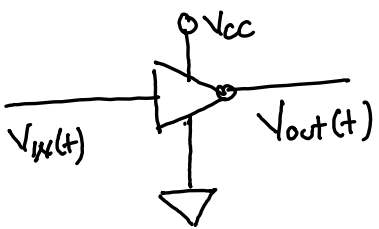
We can implement any logic function using {NOT, OR} or {NOT, AND}

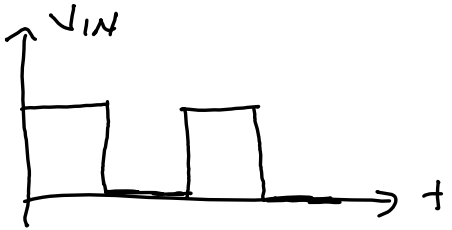
high voltage represents "1"
low voltage represents "0"

Ideal Inverter

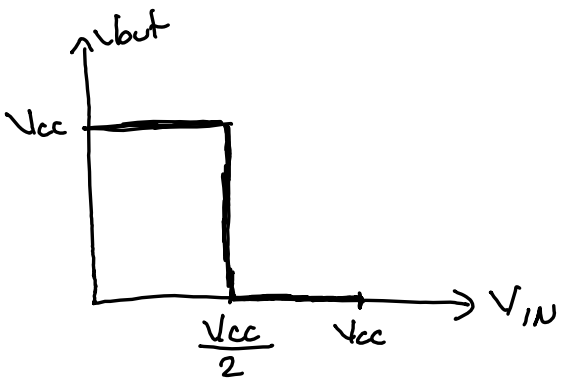
↳ A basic building block in digital electronics.



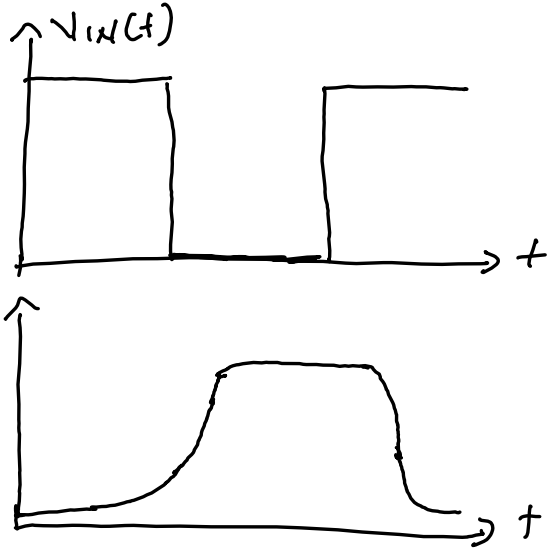






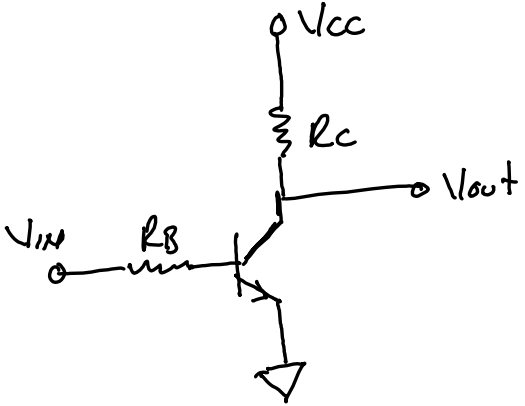


In Reality



- Related to charge removal at PN junctions
- Charging / discharging of capacitors

BJT Inverter

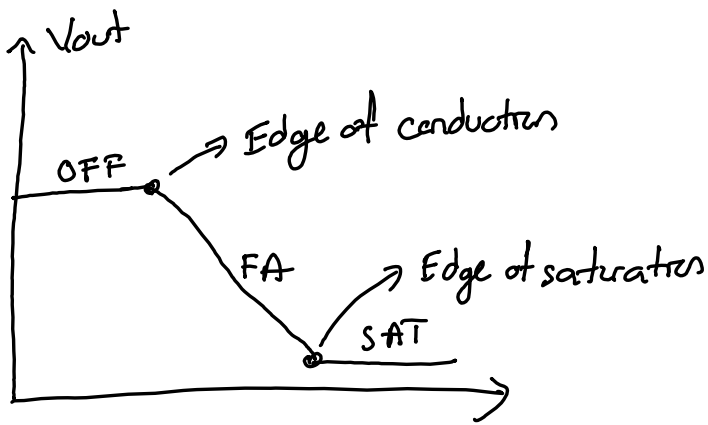


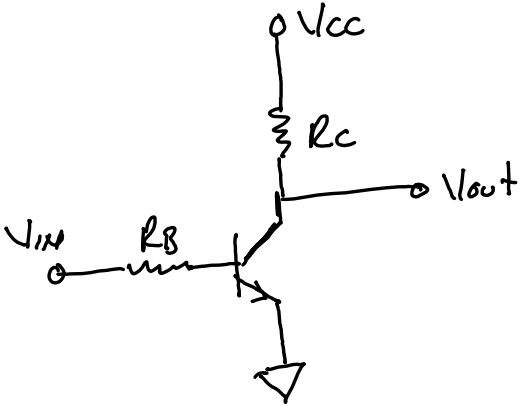
$V_{BE}(ON) = 0,7\ V$

$V_{CE}(SAT) = 0,2V$

$V_{BE}(SAT) = 0,8V$

$\beta = 100$



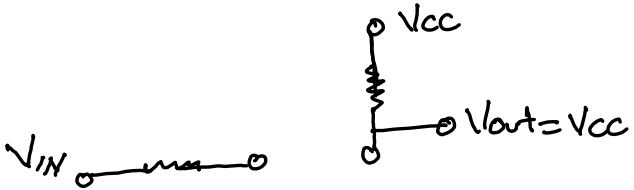


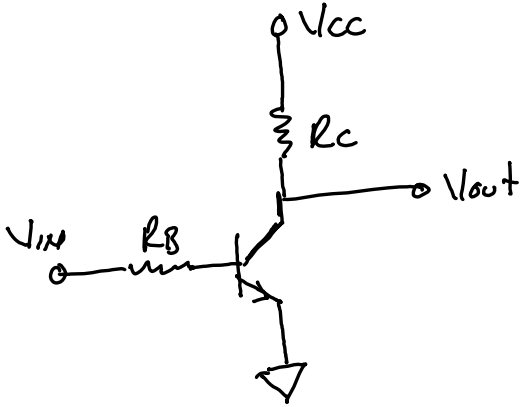
OFF Case

V_{IN} small

when $V_{IN} < V_{BE(on)}$

θ : OFF





Transistor turns on when $V_{IN} > V_{BE(ON)}$

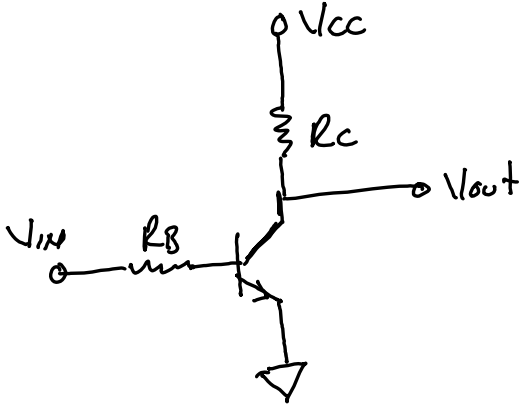
when it is on the edge

$V_{IN} = V_{BE(ON)} \rightarrow FA$

$V_{out} = V_{CC} - (\beta I_B) \cdot R_C$

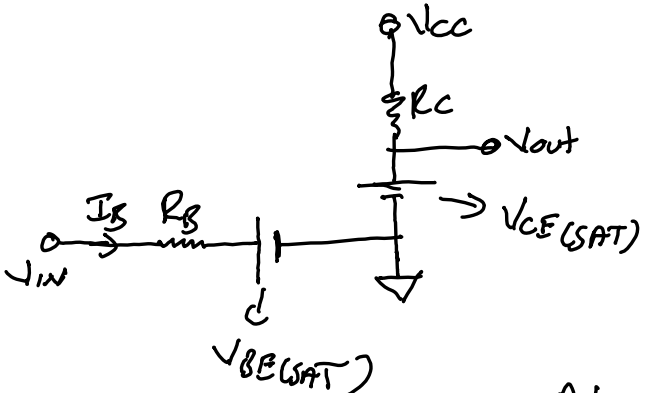
As I_B increases,

V_{out} decreases linearly



When V_{IN} is large,

Assume SAT and find V_{IN} so that SAT mode is guaranteed



$$I_B = \frac{V_{IN} - V_{BE(SAT)}}{R_B}$$

$$I_C = \frac{V_{CC} - V_{CE(SAT)}}{R_C}$$

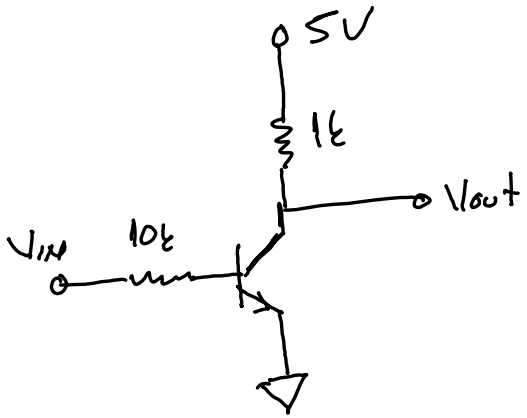
Condition for SAT : $\beta I_B > I_C$

At the edge of SAT

$$\beta \cdot \frac{V_{IN} - V_{BE(SAT)}}{R_B} = \frac{V_{CC} - V_{CE(SAT)}}{R_C}$$

$$V_{IN} = \frac{R_B}{R_C \cdot \beta}(V_{CC} - V_{CE(SAT)}) + V_{BE(SAT)}$$

(at the edge)



$\beta = 100$

$V_{BE(ON)} = 0.7V$

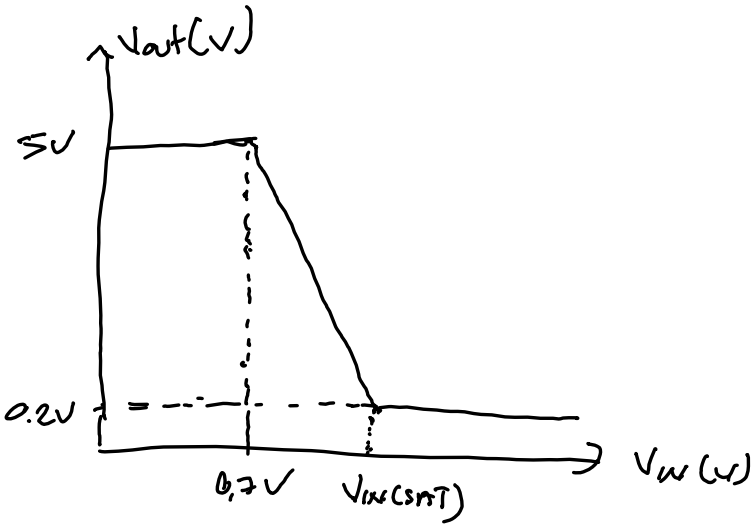
$V_{CE(SAT)} = 0.2V$

$V_{BE(SAT)} = 0.8V$

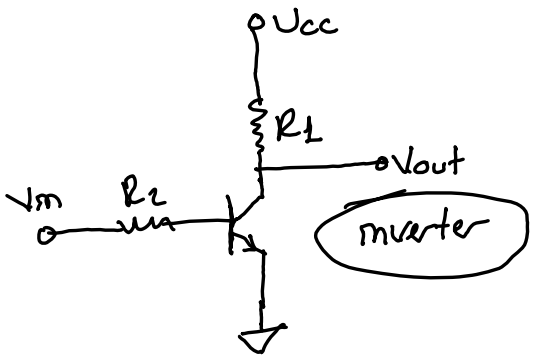
$$V_{IN(SAT)} = \frac{V_{CC} - V_{CE(SAT)}}{R_C \cdot \beta} \cdot R_B + V_{BE(SAT)}$$

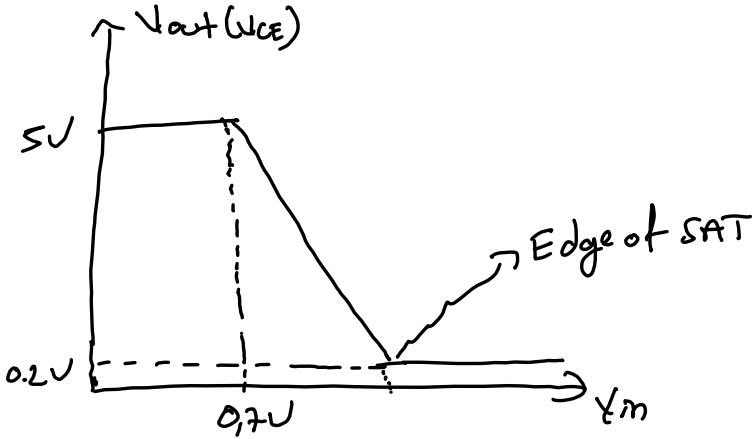
$$= \frac{5V - 0.2V}{1k \cdot 100} \cdot 10k + 0.8V$$

$$= 1.28V$$



Resistor Transistor Logic (RTL)

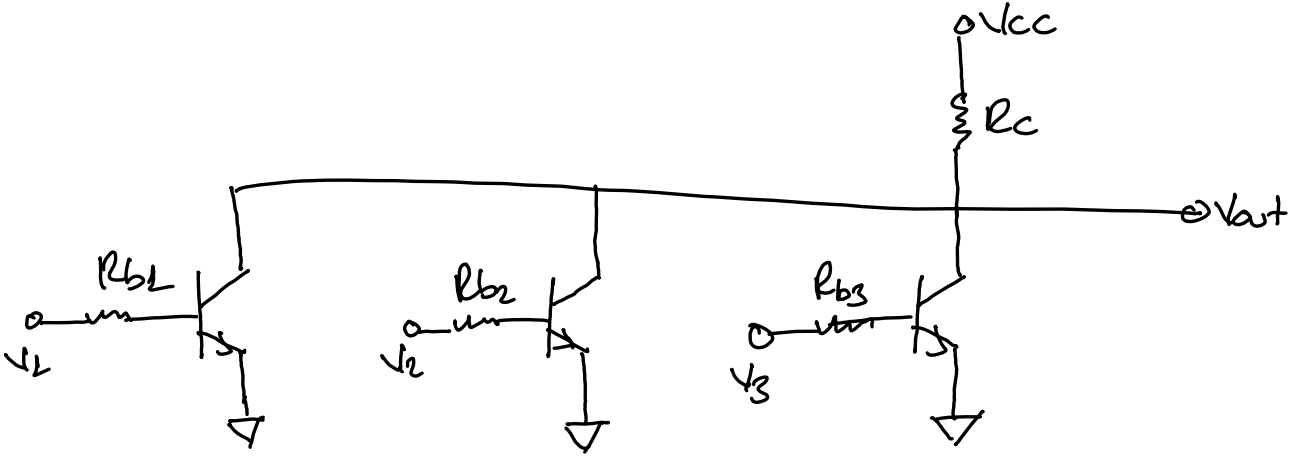




$$I_C = \frac{V_{DD} - V_{CE(SAT)}}{R_1}$$

$$I_B = \frac{I_C}{\beta} \text{ (at the edge)} \Rightarrow V_{IN} = 0.8\,V + R_2 \cdot I_B = 0,8\,V + \frac{4.8}{R_1} \cdot \frac{R_2}{\beta}$$

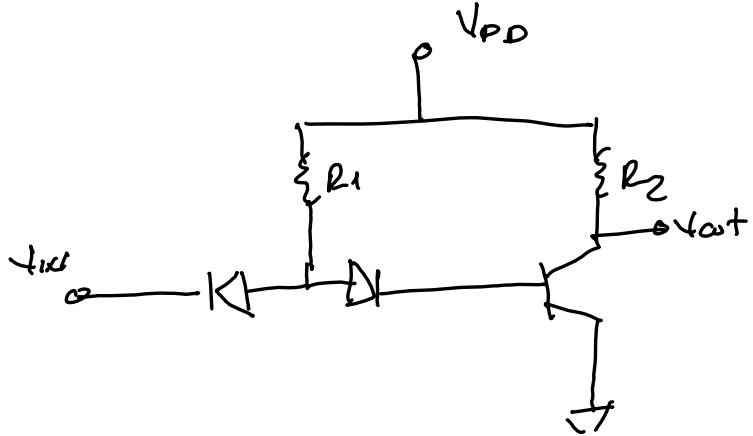
RTL NOR

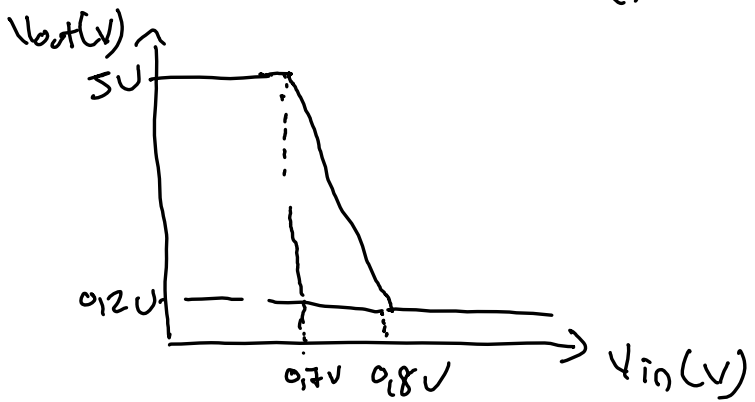


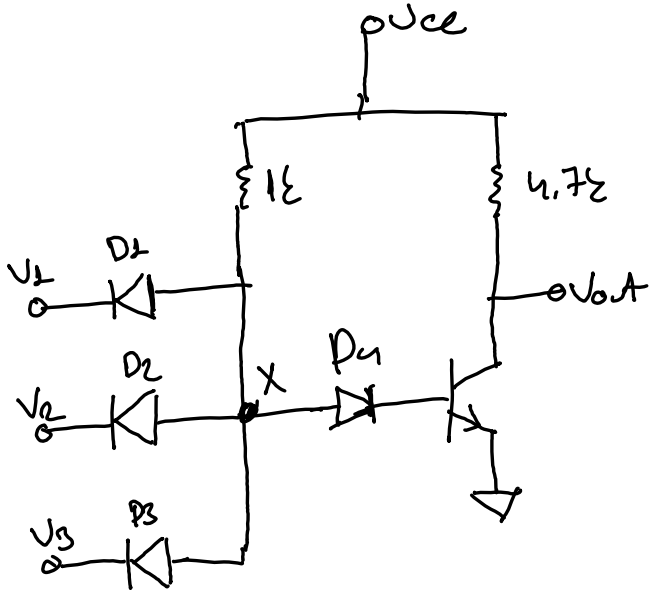
if one of V_1, V_2 or V_3 is high

then Vout is low

Diode Transistor Logic







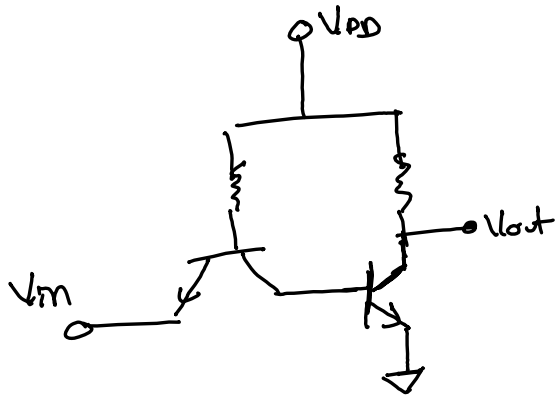
NAND

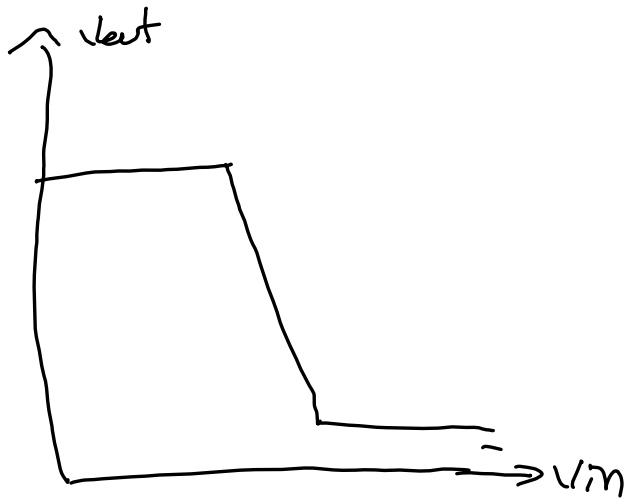
if V_1 is low, D1 is on

X's potential is not high enough to turn D4 & BJT on. Hence Vout will be high.

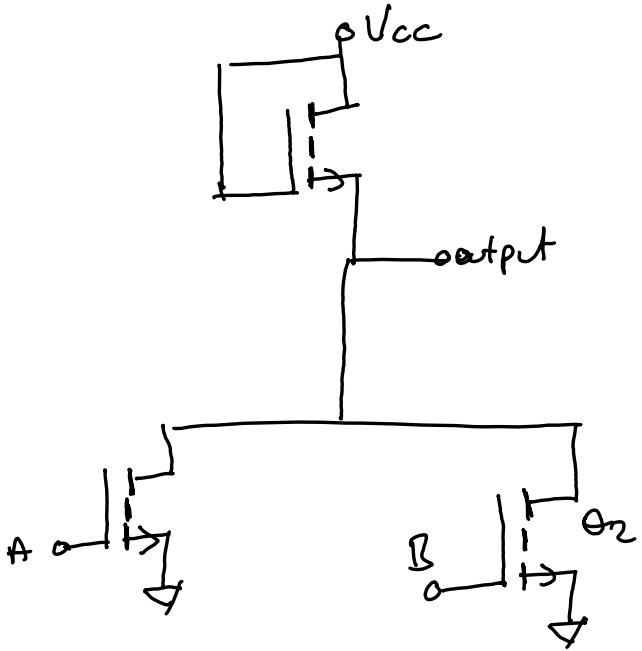
if V_1 & V_2 & V_3 are high, D1 & D2 & D3 are off. Vx will be high enough to saturate the BJT. Hence Vout will be low

Transistor - Transistor Logic (TTL)





NMOS NOR



if A & B are Low

then

Q_1 & Q_2 will be OFF

output will be pulled HIGH by Q_3

if either A or B is HIGH

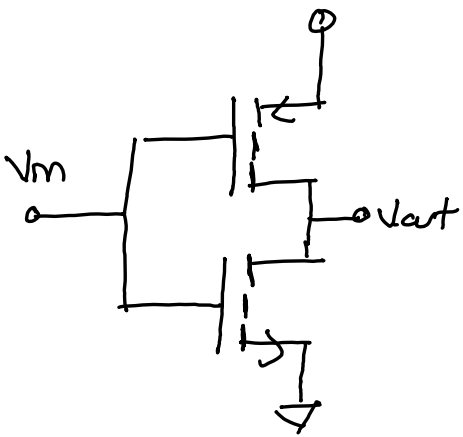
then output will go low

CMOS Logic

CMOS: Complementary Metal Oxide Semiconductor

One of the most popular structure for implementing logic gates

- faster than NMOS
- zero static power dissipation



When Vin is HIGH

NMOS Linear, PMOS OFF

⇓

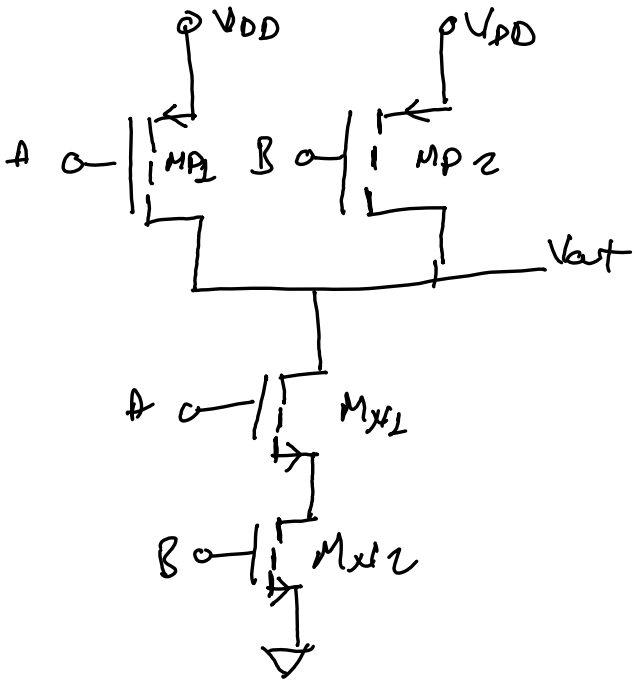
Output is Low

When Vin is LOW

NMOS is OFF, PMOS Linear

⇒ output is HIGH

CMOS NAND



If both A and B are High,

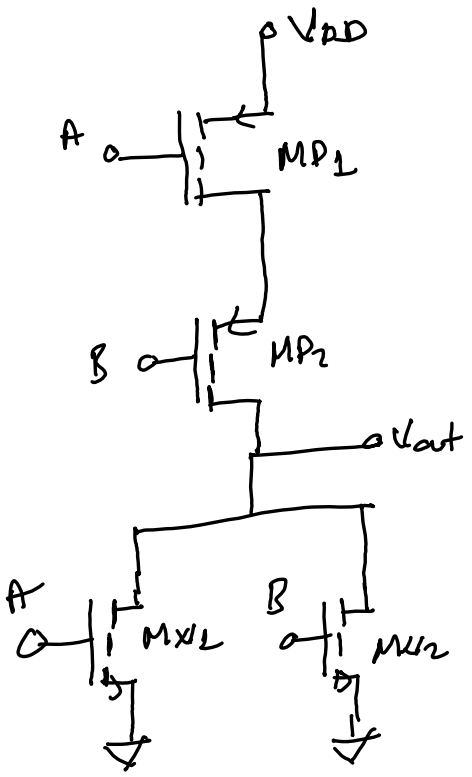
M_{N_1} & M_{N_2} ON
M_{P_1} & M_{P_2} OFF
} output Low

If either A or B or both Low

M_{N_1} or M_{N_2} or both OFF

Output HIGH

CMOS NOR



if either A or B or both high

MN1 or MN2 or both ON

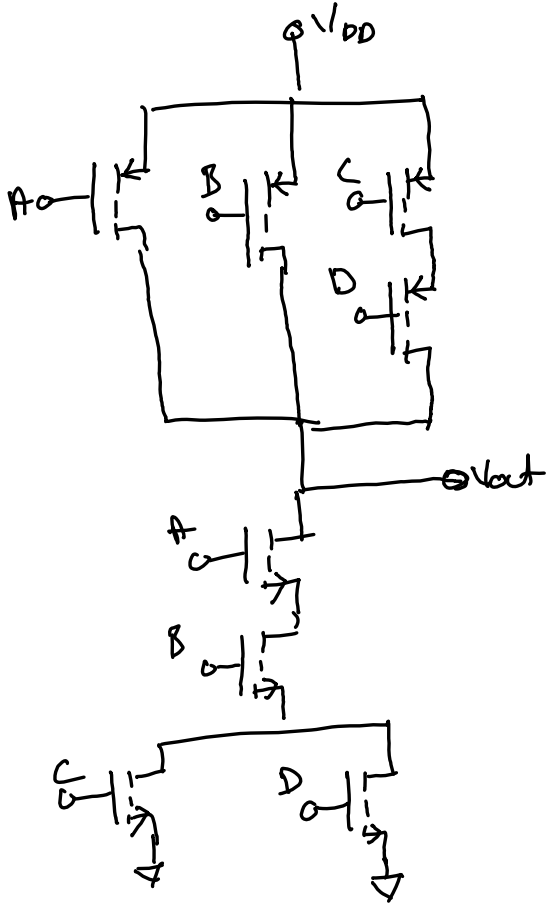
Output Low

if both A and B are Low

MN1 and MN2 OFF } output
MP1 and MP2 ON } HIGH

EX $\overline{A\,B\,(C+D)}$

	Product	Sum
Pmos	Parallel	Series
Nmos	Series	Parallel

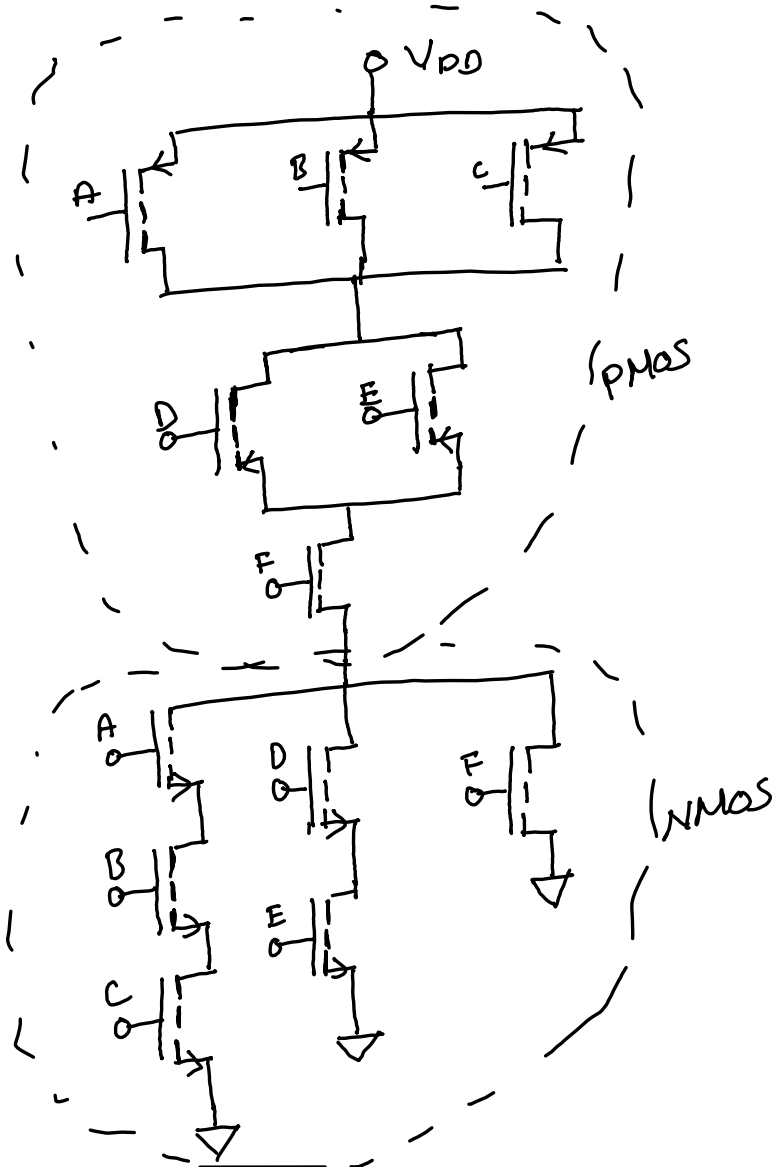


How to implement the function $\overline{ABC+DE+F}$ using CMOS transistors?

	product	sum
PMOS	parallel	series
NMOS	series	parallel

(P) product (P) arallel

(P) um (S) eries



$\overline{ABC+DE+F}$

Ex: $\overline{A'B(C+A)+B'+C}$ Implement the functres using CMOS transisters

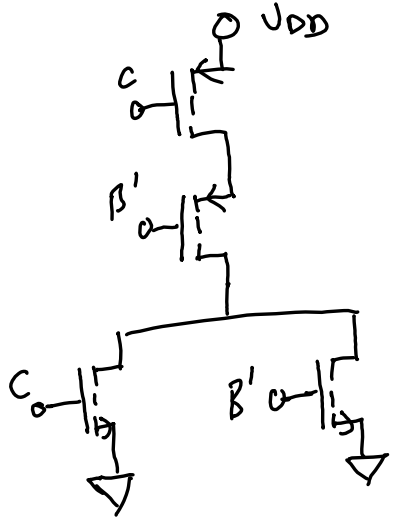
Assume that complements of inputs are available

Answer: $\overline{A'B(C+A)+B'+C} = \overline{A'BC + A'BA + B' + C}$

($A'BA = 0$)

$= \overline{C(A'B+1)+B'} = \overline{C+B'}$

($A'B + 1 = 1$)



Ex// Implement $z = AB + C'$ using two stage CMOS.

Note that inverse of the functions are not available

Ex// Implement $z = AB + C'$ using two stage CMOS.

Note that inverse of the functions are not available

$z = \overline{\overline{AB + C'}} = \overline{\overline{(AB).C}}$

Implement $\overline{AB}$ first $\xrightarrow{\text{then}}$ $\overline{\overline{AB}.C}$

